Lab 8 - Naive Bayes on Play Tennis dataset

In [1]:
```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classificatior
```

In [2]:
```
df = pd.read_csv("Lab 8 - Sheet1.csv")
df.head()
```

Out[2]:

	No	Outlook	Temperature	Humidity	Wind	Play Tennis
0	1	Sunny	Hot	High	Weak	No
1	2	Sunny	Hot	High	Strong	No
2	3	Overcast	Hot	High	Weak	Yes
3	4	Rain	Mild	High	Weak	Yes
4	5	Rain	Cool	Normal	Weak	Yes

In [3]:
```
df.shape
```

Out[3]:
```
(50, 6)
```

In [4]:
```
df["Play Tennis"].value_counts()
```

Out[4]:
```
Play Tennis
Yes    34
No     16
Name: count, dtype: int64
```

Dataset has 50 rows and 6 columns. Yes appears 34 times and No 16 times, so the data is not balanced and Yes is almost twice as common.

Features and target

In [5]:
```
X = df[["Outlook", "Temperature", "Humidity", "Wind"]]
y = df["Play Tennis"]
X.head()
```

Out[5]:

	Outlook	Temperature	Humidity	Wind
0	Sunny	Hot	High	Weak
1	Sunny	Hot	High	Strong
2	Overcast	Hot	High	Weak
3	Rain	Mild	High	Weak
4	Rain	Cool	Normal	Weak

Encoding

In [6]:
```
enc = OrdinalEncoder()
X_enc = enc.fit_transform(X)
le = LabelEncoder()
y_enc = le.fit_transform(y)
print(enc.categories_)
print(le.classes_)
```

```
[array(['Overcast', 'Rain', 'Sunny'], dtype=object), array(['Cool', 'Hot',
'Mild'], dtype=object), array(['High', 'Normal'], dtype=object), array(['Str
ong', 'Weak'], dtype=object)]
['No' 'Yes']
```

In [7]:
```
pd.DataFrame(X_enc, columns=X.columns).head()
```

Out[7]:

	Outlook	Temperature	Humidity	Wind
0	2.0	1.0	0.0	1.0
1	2.0	1.0	0.0	0.0
2	0.0	1.0	0.0	1.0
3	1.0	2.0	0.0	1.0
4	1.0	0.0	1.0	1.0

OrdinalEncoder gives a number to each category in alphabetical order, for example Outlook becomes Overcast 0, Rain 1, Sunny 2. Target is encoded as No 0 and Yes 1. CategoricalNB needs the features as whole numbers like this.

Train test split

In [8]:
```
X_train, X_test, y_train, y_test = train_test_split(X_enc, y_enc, test_size=
print(X_train.shape, X_test.shape)
```

```
(35, 4) (15, 4)
```

Categorical Naive Bayes

In [9]:
```
nb = CategoricalNB()
nb.fit(X_train, y_train)
y_pred = nb.predict(X_test)
y_pred
```

Out[9]:
```
array([0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1])
```

In [10]:
```
acc_nb = accuracy_score(y_test, y_pred)
print("Accuracy:", acc_nb)
```

```
Accuracy: 0.8
```

In [11]:
```
print(confusion_matrix(y_test, y_pred))
```

```
[[ 1  1]
 [ 2 11]]
```

In [12]:
```
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

```
              precision    recall  f1-score   support

          No       0.33      0.50      0.40         2
         Yes       0.92      0.85      0.88        13

    accuracy                           0.80        15
   macro avg       0.62      0.67      0.64        15
weighted avg       0.84      0.80      0.82        15
```

Accuracy on the test set is 0.8, so 12 of the 15 test rows are correct. Confusion matrix shows 1 correct No, 11 correct Yes, 2 Yes rows predicted as No and 1 No row predicted as Yes. Recall for Yes is 0.85 but for No it is only 0.50, because the test set has just 2 No rows and the model leans towards Yes.

Prediction for new condition

In [13]:
```
new = pd.DataFrame([["Sunny", "Cool", "High", "Strong"]], columns=X.columns)
new_enc = enc.transform(new)
pred = nb.predict(new_enc)
prob = nb.predict_proba(new_enc)
print("Predicted class:", le.inverse_transform(pred)[0])
print("Probabilities:", dict(zip(le.classes_, prob[0])))
```

```
Predicted class: No
Probabilities: {'No': np.float64(0.9372988609654674), 'Yes': np.float64(0.06
270113903453266)}
```

For Sunny, Cool, High, Strong the model predicts No with probability 0.937 and Yes only 0.063. This matches the data, since sunny days with high humidity are mostly No.

Comparison with Decision Tree and Logistic Regression

In [14]:
```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
acc_dt = accuracy_score(y_test, dt_pred)

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)
acc_lr = accuracy_score(y_test, lr_pred)

print("Naive Bayes:", acc_nb)
print("Decision Tree:", acc_dt)
print("Logistic Regression:", acc_lr)
```

```
Naive Bayes: 0.8
Decision Tree: 0.6
Logistic Regression: 0.5333333333333333
```

In [15]:
```
print(classification_report(y_test, dt_pred, target_names=le.classes_))
print(classification_report(y_test, lr_pred, target_names=le.classes_))
```

```
              precision    recall  f1-score   support

          No       0.17      0.50      0.25         2
         Yes       0.89      0.62      0.73        13

    accuracy                           0.60        15
   macro avg       0.53      0.56      0.49        15
weighted avg       0.79      0.60      0.66        15

              precision    recall  f1-score   support

          No       0.00      0.00      0.00         2
         Yes       0.80      0.62      0.70        13

    accuracy                           0.53        15
   macro avg       0.40      0.31      0.35        15
weighted avg       0.69      0.53      0.60        15
```

Naive Bayes gives 0.80, Decision Tree 0.60 and Logistic Regression 0.533. Naive Bayes works best here because all features are categorical and it handles them directly. Decision Tree overfits the small 35 row training set. Logistic Regression treats the encoded numbers as if they have an order, which is wrong for values like Outlook, and it never predicts No at all so its precision and recall for No are both 0.